

Servlets, Servlet Container & Tomcat — Notes

1. What is a Servlet?

Definition: A Servlet is a Java class that runs on a server and handles HTTP requests and responses. It is part of Java EE (Jakarta EE) specification and forms the base for building web applications in Java.

Key Points:

- Servlets extend the capability of a server to handle client requests dynamically.
- They run inside a **Servlet Container** (not standalone).
- Every servlet must implement the `javax.servlet.Servlet` interface (directly or via `HttpServlet`).
- Servlets work on a **request-response model**.

Servlet Lifecycle (very important):

1. **Loading & Instantiation** — Container loads the servlet class and creates an instance.
2. **init()** — Called once, when the servlet is first loaded. Used for initialization (DB connections, config, etc.)
3. **service()** — Called for every request. Internally dispatches to `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc. based on HTTP method.
4. **destroy()** — Called once, when the servlet is being taken out of service (app shutdown/undeploy). Used for cleanup.

```
Client Request → Container → init() [only once]
                               → service() → doGet()/doPost() [for every request]
                               → destroy() [only once, at shutdown]
```

2. What is a Servlet Container?

Definition: A Servlet Container (also called a Web Container) is a part of the web server that manages the lifecycle of servlets, maps URLs to specific servlets, and ensures proper request/response handling.

Examples of Servlet Containers: Apache Tomcat, Jetty, Undertow, WildFly (JBoss)

Responsibilities of a Servlet Container:

- **Lifecycle Management** — Creates, initializes, invokes, and destroys servlet instances.
- **Communication Support** — Handles low-level socket communication and converts raw HTTP data into `HttpServletRequest` and `HttpServletResponse` Java objects.
- **Multithreading** — Manages a thread pool so multiple requests can be served concurrently

without creating a new servlet instance for each request.

- **URL Mapping** — Maps incoming request URLs to the correct servlet based on `web.xml` or annotations (`@WebServlet`).
 - **Security** — Handles authentication/authorization as configured.
 - **Session Management** — Manages `HttpSession` objects for tracking users across requests.
-

3. How This Fits into the Spring Framework

Spring (specifically **Spring MVC**) is built **on top of** the Servlet API — it does not replace it.

Key Component: DispatcherServlet

- `DispatcherServlet` is itself a **servlet** (it extends `HttpServlet`) — so it must run inside a servlet container like Tomcat.
- It implements the **Front Controller Design Pattern** — a single entry point for all incoming requests to a Spring web application.

Flow of Request Processing in Spring MVC:

```
1. Client sends HTTP Request
   ↓
2. Request reaches Tomcat (Servlet Container)
   ↓
3. Tomcat forwards request to DispatcherServlet
   (since it is registered as the front controller in web.xml / auto-configured
   in Spring Boot)
   ↓
4. DispatcherServlet consults HandlerMapping
   to find which Controller should handle this URL
   ↓
5. Request is sent to the appropriate Controller (@Controller / @RestController)
   ↓
6. Controller processes business logic (often calls Service → Repository layers)
   ↓
7. Controller returns a ModelAndView (or data, in case of REST)
   ↓
8. DispatcherServlet sends this to ViewResolver
   (to resolve JSP/Thymeleaf page) – only for traditional MVC, not REST
   ↓
9. View is rendered and Response sent back through DispatcherServlet
   ↓
10. Response goes back through Tomcat to the Client
```

Why this matters:

- Spring **relies on the servlet container** to actually receive requests from the network — Spring doesn't handle sockets/networking itself.
- `DispatcherServlet` acts as the bridge between the low-level servlet world and Spring's high-level MVC architecture (Controllers, Services, etc.)
- In **Spring Boot**, an **embedded Tomcat** is automatically bundled and configured, so you

don't need to deploy a WAR file separately — the container starts along with your application (`main()` method).

4. Apache Tomcat — Detailed Explanation

Definition: Tomcat is an open-source **web server and servlet container** developed by the Apache Software Foundation. It implements the Java Servlet, JSP, and WebSocket specifications.

Core Architecture of Tomcat:

Component	Role
Server	Represents the entire Tomcat instance
Service	Groups a Connector + Engine together
Connector	Handles communication (HTTP/1.1, HTTP/2, AJP protocols); listens on a port (default 8080)
Engine	Represents request processing pipeline for a Service
Host	Represents a virtual host (like a domain)
Context	Represents a single web application (maps to a <code>.war</code> file or folder)
Wrapper	Represents an individual servlet within a Context

How Tomcat Processes a Request (step-by-step):

1. Client sends an HTTP request to the server's IP/port.
2. **Connector** accepts the raw TCP/HTTP request.
3. Request is passed to the **Engine**, which routes it to the correct **Host** (based on domain name).
4. Host routes it to the correct **Context** (based on application/context path, e.g., `/myapp`).
5. Context finds the right **Wrapper** (servlet) based on URL pattern.
6. Tomcat creates `HttpServletRequest` and `HttpServletResponse` objects.
7. The servlet's `service()` method is invoked with these objects.
8. Response is generated and sent back through the same pipeline to the client.

Key Features of Tomcat:

- Lightweight and widely used for Java web applications.
- Supports **multithreading** — a thread pool handles concurrent requests efficiently.
- Can run standalone (deploy `.war` files) or **embedded** (as in Spring Boot apps).
- Configurable via `server.xml`, `web.xml`, and `context.xml`.

- Supports HTTPS/SSL configuration, connection pooling, clustering, and load balancing.

Standalone Tomcat vs Embedded Tomcat (Spring Boot):

Standalone Tomcat	Embedded Tomcat (Spring Boot)
Installed separately on the machine	Comes bundled as a dependency (<code>spring-boot-starter-web</code>)
You deploy a <code>.war</code> file into <code>webapps/</code> folder	Tomcat starts automatically when you run <code>main()</code>
Needs manual configuration (<code>server.xml</code>)	Configured via <code>application.properties/application.yml</code>
Common in traditional enterprise deployments	Common in modern microservices/Spring Boot apps

5. Quick Summary (for revision)

- **Servlet** = Java class that handles HTTP requests dynamically; has a lifecycle (`init` → `service` → `destroy`).
- **Servlet Container** = manages servlets' lifecycle, threading, and URL mapping (e.g., Tomcat).
- **Spring MVC's DispatcherServlet** = a special servlet acting as a Front Controller; sits between Tomcat and your application's Controllers.
- **Tomcat** = the actual servlet container + web server that receives requests from the network and hands them to DispatcherServlet.
- **Spring Boot** simplifies this by embedding Tomcat directly inside the application — no separate deployment needed.